MANUAL DE PREPARACIÓN DE EXAMEN

TypeScript + React

Guía de estudio práctica para desarrollo web en entorno cliente: conceptos, errores típicos, ejercicios resueltos y simulacros.

01 Fundamentos
Tipos, inferencia, uniones, narrowing y genéricos.

02 React
JSX, props, estado, eventos, formularios y efectos.

03 Práctica
8 ejercicios, 2 simulacros resueltos y chuleta final.

Edición mejorada y corregida · octubre de 2026
Contrastada con la documentación oficial de React, TypeScript y Vite.

Contenido

Cómo usar esta guía 3

1. Mapa de prioridades 3

2. TypeScript esencial 4

2.1. Qué es y qué no hace 4
2.2. Tipos básicos 4
2.3. Inferencia 4
2.4. Objetos, arrays y funciones 4
2.5. Uniones, literales y propiedades opcionales 5
2.6. Narrowing: comprobar antes de usar 5
2.7. interface o type 5
2.8. Genéricos y utility types 5
2.9. tsconfig.json 6

3. React esencial 6

3.1. Qué es React y cómo actualiza la pantalla 6
3.2. Crear un proyecto con Vite 6
3.3. main.tsx 6
3.4. Reglas de JSX 6
3.5. Props 6
3.6. children 7
3.7. Renderizado condicional 7
3.8. Listas y key 7

4. Estado con useState 8

4.1. Por qué no basta una variable normal 8
4.2. El estado es una «foto» por render 8
4.3. Tipar el estado 8
4.4. Inmutabilidad 8
4.5. Datos derivados 9

5. Eventos y formularios 9

5.1. Pasar la función, no ejecutarla 9
5.2. Tipos de evento más frecuentes 9
5.3. preventDefault y stopPropagation 9
5.4. Formulario controlado 9
5.5. Checkbox, select y varios campos con un solo objeto 10
5.6. target y currentTarget 10

6. Elevar el estado (lifting state up) 10

7. useEffect y useRef (nivel básico) 11

8. Errores típicos y cómo corregirlos 11

9. Conceptos que se confunden 12

10. Ejercicios resueltos (de menor a mayor dificultad) 12

Ejercicio 1. Contador que no baja de cero 12
Ejercicio 2. Contador de caracteres derivado 13
Ejercicio 3. Lista de tareas con formulario controlado 13
Ejercicio 4. Búsqueda con estado elevado 14
Ejercicio 5. Tareas con completar, borrar y contador de pendientes 15
Ejercicio 6. Formulario con validación y mensajes de error 16
Ejercicio 7. Carrito con callback y total derivado 17
Ejercicio 8 (extra). Cargar datos con fetch, cargando y error 17

11. Resumen y checklist 18

12. Simulacros de examen 18

Simulacro 1 (del material original, con corrección ampliada) 18
Simulacro 2 (nuevo, 45 minutos) 19
Soluciones del simulacro 2 19

13. Estrategia para el día del examen 21

Chuleta de una página 22

Referencias oficiales 22

Cómo usar esta guía

Esta edición conserva todo lo que ya tenías (tipos, props, `useState`, eventos, formularios, inmutabilidad y elevación del estado), corrige varios detalles técnicos y añade lo que suele aparecer en exámenes de este nivel: narrowing, genéricos, renderizado condicional, formularios con varios campos, `useEffect`, más ejercicios resueltos, dos simulacros y una chuleta final de una página.

> **La idea central.** React describe la interfaz; el estado guarda los datos que cambian; las props llevan datos del padre al hijo; TypeScript comprueba que los valores y funciones tienen los tipos esperados.

Ruta recomendada

1. Lee los capítulos 1 a 6 y teclea cada ejemplo en un proyecto real.
2. Estudia los errores típicos (cap. 8) y la tabla de conceptos (cap. 9).
3. Resuelve los ejercicios del cap. 10 **sin mirar la solución**.
4. Haz los dos simulacros (cap. 12) con tiempo limitado.
5. La noche anterior, repasa solo la chuleta final.

Qué se ha corregido respecto al material original

Cambio	Detalle
Corregido	La tabla de tipos de eventos perdía los genéricos (`ChangeEvent<HTMLInputElement>`). Aquí aparece completa.
Corregido	En el simulacro, `contador = contador + 1` no es solo un error de React: TypeScript también lo marca, porque `contador` es `const`.
Matizado	`Date.now()` como `id` funciona en clase, pero puede repetirse si se añaden dos elementos en el mismo milisegundo. Se explican alternativas.
Añadido	Narrowing, genéricos, utility types, renderizado condicional, checkbox y select, formulario con objeto de estado, `useEffect` con `fetch`, `useRef`.
Añadido	Ejercicios 5 a 8, simulacro 2 con soluciones, estrategia de examen y chuleta.

1. Mapa de prioridades

Tema	Frecuencia	Qué debes saber hacer
`useState` e inmutabilidad	Muy alta	Contadores, listas, objetos; añadir, borrar y modificar sin mutar
Props tipadas	Muy alta	Declarar el tipo, props opcionales, callbacks, `children`
Formularios controlados	Muy alta	`value` + `onChange` + `onSubmit` + `preventDefault` + validación
Listas con `map` y `key`	Muy alta	`key` estable, filtrar y ordenar sin mutar
Elevar el estado	Alta	Estado en el ancestro común, callbacks hacia arriba
Eventos tipados	Alta	`ChangeEvent`, `FormEvent`, `MouseEvent` con su genérico
Datos derivados	Alta	Calcular en el render en vez de duplicar estado
Renderizado condicional	Alta	`&&`, ternario, retorno temprano
`useEffect` y `fetch`	Media	Carga inicial, estados `cargando` y `error`
Uniones, literales, narrowing	Media	Tipos con varias opciones, comprobar `undefined`
Genéricos y utility types	Baja	`useState<T[]>`, `Partial`, `Omit`

2. TypeScript esencial

2.1. Qué es y qué no hace

TypeScript añade un sistema de tipos a JavaScript. Los tipos **desaparecen** al compilar, así que no validan en tiempo de ejecución los datos de un formulario o de una API.

> **Ojo con Vite.** En un proyecto Vite, el servidor de desarrollo transpila TypeScript pero no siempre comprueba los tipos. Para comprobarlos de forma explícita usa `npx tsc --noEmit`.

2.2. Tipos básicos

Tipo	Uso	Ejemplo
`number`	Enteros y decimales	`let nota: number = 8.5;`
`string`	Texto	`let nombre: string = "Ana";`
`boolean`	Verdadero o falso	`let ok: boolean = true;`
`any`	Desactiva la comprobación. Evítalo.	`let x: any = 10;`
`unknown`	Valor desconocido; obliga a comprobar antes de usarlo	`let x: unknown = "hola";`
`void`	Función sin valor de retorno útil	`function avisar(): void {}`
`never`	Valor imposible o función que no termina	`function fallar(): never { throw new Error("x"); }`
`null`, `undefined`	Ausencia de valor	`let u: string | null = null;`

2.3. Inferencia

Si asignas un valor inicial, TypeScript deduce el tipo. Escribe el tipo explícito cuando aclare el contrato (parámetros, retorno, props) o cuando TypeScript no pueda inferirlo (por ejemplo, un array vacío).

```
let puntos = 10;            // number (inferido)
const ciudad = "Oviedo";    // tipo literal "Oviedo"
// puntos = "diez";         // Error de tipo
const lista: string[] = []; // sin tipo, un array vacío no se puede inferir bien
```

2.4. Objetos, arrays y funciones

```
type Developer = { nombre: string; edad: number; activo: boolean };

const ana: Developer = { nombre: "Ana", edad: 22, activo: true };
const lenguajes: string[] = ["React", "TypeScript"];
lenguajes.push("Vite");     // Correcto
// lenguajes.push(42);      // Error

const multiplicar = (a: number, b: number): number => a * b;
```

Otras formas de escribir arrays: `Array<string>`. Tuplas (longitud y tipos fijos): `let par: [string, number] = ["edad", 22];`.

2.5. Uniones, literales y propiedades opcionales

```ts
let codigo: string | number = "ID-402";
codigo = 402;

type NivelAlerta = "info" | "aviso" | "error";
let alerta: NivelAlerta = "aviso";
// alerta = "otro";         // Error: no está en la lista

interface Usuario {
  nombre: string;
  edad?: number;            // number | undefined
}
```

2.6. Narrowing: comprobar antes de usar

Cuando un valor puede ser de varios tipos, TypeScript te obliga a **acotarlo** antes de operar con él.

```ts
function siguienteEdad(u: Usuario): number | string {
  if (u.edad !== undefined) {
    return u.edad + 1;      // aquí edad es number
  }
  return "sin edad";
}

const edadSegura = usuario.edad ?? 0;      // ?? solo sustituye null y undefined
const longitud = usuario.nombre?.length;   // ?. evita el error si es null o undefined

function mostrar(valor: string | number) {
  if (typeof valor === "string") return valor.toUpperCase();
  return valor.toFixed(2);
}

function leer(dato: unknown) {
  if (typeof dato === "string") return dato.length; // unknown exige comprobar
  return 0;
}
```

2.7. `interface` o `type`

Característica	`interface`	`type`
Describir la forma de un objeto	Sí	Sí
Uniones y literales	No	Sí
Ampliar	`extends`	Intersección `&`
Declarar dos veces el mismo nombre	Se fusionan	Error

Para props de React ambos valen. Usa el que pida tu profesor; `type` es más cómodo cuando hay uniones.

2.8. Genéricos y utility types

Un genérico es un tipo «con hueco» que se rellena al usarlo. Lo verás sobre todo en `useState<T>`.

```ts
function primero<T>(lista: T[]): T | undefined {
  return lista[0];
}
const n = primero([1, 2, 3]);        // number | undefined

type Tarea = { id: number; texto: string; hecha: boolean };

type TareaNueva = Omit<Tarea, "id">;           // todas menos id
type CambiosTarea = Partial<Tarea>;            // todas opcionales
type SoloTexto = Pick<Tarea, "id" | "texto">;  // solo esas dos
type Conteo = Record<string, number>;          // claves string, valores number
```

2.9. `tsconfig.json`

Los archivos `.ts` contienen TypeScript y los `.tsx` además permiten JSX. Opciones clave: `strict` (comprobaciones estrictas, recomendable), `jsx` (cómo se transforma el JSX) y `noEmit` (no generar JavaScript al comprobar tipos).

3. React esencial

3.1. Qué es React y cómo actualiza la pantalla

React construye la interfaz con componentes: funciones que reciben datos y devuelven JSX. Cuando cambian el estado o las props, React vuelve a ejecutar el componente, compara la nueva descripción con la anterior y actualiza solo lo necesario del DOM real. La pantalla no cambia porque modifiques una variable cualquiera.

3.2. Crear un proyecto con Vite

```
npm create vite@latest mi-app -- --template react-ts
cd mi-app
npm install
npm run dev
```

3.3. `main.tsx`

```tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.tsx";

const root = document.getElementById("root");
if (!root) throw new Error("No se encontró #root");

createRoot(root).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

> **StrictMode** solo actúa en desarrollo: puede renderizar una vez extra para detectar funciones impuras y volver a ejecutar efectos para comprobar su limpieza. En producción no duplica nada.

3.4. Reglas de JSX

- Un solo elemento raíz; si no quieres una etiqueta extra, usa un fragmento `<> ... </>`.
- Cierra todas las etiquetas: `<input />`.
- `className` en vez de `class`; eventos en camelCase (`onClick`, `onChange`).
- Las llaves `{ }` insertan expresiones JavaScript (no sentencias como `if` o `for`).
- `style` recibe un objeto: `style={{ padding: "20px" }}`.
- Los componentes empiezan por mayúscula; si empiezan por minúscula, React los trata como etiquetas HTML.

3.5. Props

Las props viajan del padre al hijo y son de **solo lectura**. Si el hijo necesita avisar al padre, recibe una función por props y la llama.

```tsx
type TarjetaProps = {
  titulo: string;
  precio: number;
  disponible?: boolean;
};

function TarjetaProducto({ titulo, precio, disponible = true }: TarjetaProps) {
  return (
    <article>
      <h2>{titulo}</h2>
      <p>Precio: {precio} €</p>
      <p>{disponible ? "Disponible" : "Agotado"}</p>
    </article>
  );
}

// Uso:
<TarjetaProducto titulo="Teclado" precio={29.99} />
```

Los textos van con comillas (`titulo="Teclado"`); números, booleanos y objetos, con llaves (`precio={29.99}`).

3.6. `children`

```tsx
import type { ReactNode } from "react";

type AvisoProps = {
  titulo: string;
  children: ReactNode;
  tipo?: "info" | "aviso";
};

function Aviso({ titulo, children, tipo = "info" }: AvisoProps) {
  return (
    <section className={`aviso aviso-${tipo}`}>
      <strong>{titulo}</strong>
      <div>{children}</div>
    </section>
  );
}

<Aviso titulo="Atención" tipo="aviso">Revisa el formulario.</Aviso>
```

3.7. Renderizado condicional

```tsx
function Panel({ usuario }: { usuario: string | null }) {
  if (!usuario) return <p>Inicia sesión</p>;          // retorno temprano

  return (
    <div>
      <p>Hola, {usuario}</p>
      {usuario === "admin" && <button>Administrar</button>}       {/* && */}
      <p>{usuario.length > 5 ? "Nombre largo" : "Nombre corto"}</p> {/* ternario */}
    </div>
  );
}
```

> **Trampa clásica.** `{lista.length && <Lista />}` pinta un `0` en pantalla cuando la lista está vacía. Escribe `{lista.length > 0 && <Lista />}`.

3.8. Listas y `key`

```tsx
{productos.map(p => (
  <li key={p.id}>{p.nombre}</li>
))}
```

La `key` debe ser estable y única entre hermanos. Usa el `id` del dato. El índice del array solo es aceptable si la lista nunca se reordena ni se modifica. La `key` va en el elemento raíz que devuelve el `map`.

4. Estado con `useState`

4.1. Por qué no basta una variable normal

Una variable local no avisa a React de que algo cambió y además se vuelve a crear en cada render. Para un dato que cambia y debe verse, usa `useState`.

```tsx
import { useState } from "react";

function Contador() {
  const [contador, setContador] = useState(0);

  return (
    <div>
      <p>Valor: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>Sumar</button>
    </div>
  );
}
```

4.2. El estado es una «foto» por render

`contador` vale lo mismo durante todo ese render. Llamar a `setContador` programa un nuevo render, pero **no cambia la variable en la ejecución actual**.

```tsx
setContador(contador + 1);
setContador(contador + 1);
setContador(contador + 1);   // Sube 1, no 3: las tres usan la misma foto

setContador(previo => previo + 1);
setContador(previo => previo + 1);
setContador(previo => previo + 1);   // Sube 3: cada una parte del resultado anterior
```

> **Regla.** Si el nuevo valor depende del anterior, usa la forma funcional `previo => ...`.

4.3. Tipar el estado

```tsx
const [nombre, setNombre] = useState("");                     // string (inferido)
const [tareas, setTareas] = useState<Tarea[]>([]);            // array vacío: hay que tiparlo
const [usuario, setUsuario] = useState<Usuario | null>(null);   // puede no haber valor aún
const [datos, setDatos] = useState<Datos>({ nombre: "", email: "" });
```

4.4. Inmutabilidad

No modifiques arrays ni objetos del estado: crea una copia nueva y pásala al setter. Si entregas la misma referencia, React puede entender que no hay cambio.

Operación	No hagas (muta)	Haz (copia nueva)
Añadir	`lista.push(x)`	`setLista(prev => [...prev, x])`
Eliminar	`lista.splice(i, 1)`	`setLista(prev => prev.filter(t => t.id !== id))`
Modificar un elemento	`lista[i].hecha = true`	`setLista(prev => prev.map(t => t.id === id ? { ...t, hecha: !t.hecha } : t))`
Cambiar un campo	`datos.nombre = "Luis"`	`setDatos(prev => ({ ...prev, nombre: "Luis" }))`
Ordenar	`lista.sort()`	`setLista(prev => [...prev].sort())`
Objeto anidado	`d.dir.ciudad = "X"`	`setD(prev => ({ ...prev, dir: { ...prev.dir, ciudad: "X" } }))`

Métodos que mutan: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`. **Métodos que devuelven copia:** `map`, `filter`, `slice`, `concat`, spread.

4.5. Datos derivados

Si un dato se puede calcular a partir de props o de otro estado, no lo guardes en `useState`: calcúlalo en el render.

```
const [tareas, setTareas] = useState<Tarea[]>([]);
const pendientes = tareas.filter(t => !t.hecha).length;                    // derivado
const total = carrito.reduce((suma, p) => suma + p.precio * p.cantidad, 0);   // derivado
```

5. Eventos y formularios

5.1. Pasar la función, no ejecutarla

```
<button onClick={sumar}>Sumar</button>                          // correcto
<button onClick={() => eliminar(item.id)}>Eliminar</button>     // correcto con argumento
<button onClick={sumar()}>Sumar</button>                        // MAL: se ejecuta al renderizar
```

5.2. Tipos de evento más frecuentes

Situación	Tipo
Cambio en `<input>`	`ChangeEvent<HTMLInputElement>`
Cambio en `<textarea>`	`ChangeEvent<HTMLTextAreaElement>`
Cambio en `<select>`	`ChangeEvent<HTMLSelectElement>`
Envío de `<form>`	`FormEvent<HTMLFormElement>`
Clic en botón	`MouseEvent<HTMLButtonElement>`
Tecla en input	`KeyboardEvent<HTMLInputElement>`

Se importan de `"react"`: `import { type ChangeEvent, type FormEvent } from "react";`. Si escribes el manejador directamente en el JSX, TypeScript suele inferir el tipo; si lo extraes a una función aparte, tipa el parámetro.

5.3. `preventDefault` y `stopPropagation`

Método	Qué hace	Uso típico
`e.preventDefault()`	Cancela la acción por defecto del navegador	Que un formulario no recargue la página
`e.stopPropagation()`	Impide que el evento suba a los contenedores	Un botón dentro de una tarjeta clicable

5.4. Formulario controlado

Un input controlado toma su `value` del estado y su `onChange` actualiza ese mismo estado.

```tsx
import { useState, type FormEvent, type ChangeEvent } from "react";

function FormularioEmail() {
  const [email, setEmail] = useState("");

  const enviar = (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    console.log(email);
  };

  const cambiar = (e: ChangeEvent<HTMLInputElement>) => setEmail(e.target.value);

  return (
    <form onSubmit={enviar}>
      <label htmlFor="email">Correo: </label>
      <input id="email" type="email" value={email} onChange={cambiar} />
      <button type="submit">Enviar</button>
    </form>
  );
}
```

5.5. Checkbox, select y varios campos con un solo objeto

```tsx
type Datos = { nombre: string; pais: string; acepta: boolean };

function Registro() {
  const [datos, setDatos] = useState<Datos>({ nombre: "", pais: "es", acepta: false });

  const cambiarTexto = (e: ChangeEvent<HTMLInputElement | HTMLSelectElement>) => {
    const { name, value } = e.target;
    setDatos(prev => ({ ...prev, [name]: value }));
  };

  const cambiarCheck = (e: ChangeEvent<HTMLInputElement>) => {
    const { checked } = e.target;
    setDatos(prev => ({ ...prev, acepta: checked }));
  };

  return (
    <form>
      <input name="nombre" value={datos.nombre} onChange={cambiarTexto} />
      <select name="pais" value={datos.pais} onChange={cambiarTexto}>
        <option value="es">España</option>
        <option value="pt">Portugal</option>
      </select>
      <input type="checkbox" checked={datos.acepta} onChange={cambiarCheck} />
    </form>
  );
}
```

El truco `[name]: value` usa el atributo `name` del campo como clave del objeto. En un checkbox se lee `checked`, no `value`.

5.6. `target` y `currentTarget`

`target` es el elemento donde se originó el evento; `currentTarget` es el elemento que tiene el manejador. Si haces clic en un `<span>` dentro de un `<button onClick>`, `target` es el span y `currentTarget` es el botón.

6. Elevar el estado (lifting state up)

Si dos componentes necesitan compartir un dato, el estado vive en su **ancestro común más cercano**. El padre pasa el valor por props y pasa también callbacks para que los hijos le comuniquen cambios.

```
App (guarda filtro)
 ├── BarraBusqueda  → recibe valor y alCambiar
 └── ListaProductos → recibe productos y filtro
```

Si un hijo necesita **mostrar** el dato, recibe el valor. Si necesita **cambiarlo**, recibe también una función. El hijo nunca modifica las props.

7. `useEffect` y `useRef` (nivel básico)

`useEffect` ejecuta código *después* de pintar el componente. Se usa para sincronizar con algo externo: pedir datos a una API, suscribirse a un evento, cambiar el título de la página. El array de dependencias decide cuándo se ejecuta:

Dependencias	Cuándo se ejecuta
`[]`	Una vez al montar el componente
`[filtro]`	Al montar y cada vez que `filtro` cambia
sin array	Después de cada render (casi nunca es lo que quieres)

La función que devuelve el efecto es la **limpieza**: se ejecuta antes de repetir el efecto y al desmontar.

```
useEffect(() => {
  const id = setInterval(() => setSegundos(s => s + 1), 1000);
  return () => clearInterval(id);   // limpieza
}, []);
```

> **No uses `useEffect` para datos derivados.** Si puedes calcularlo en el render (un filtro, un total, una longitud), hazlo ahí.

`useRef` guarda un valor que persiste entre renders pero **no provoca un render al cambiar**. Usos habituales: acceder a un elemento del DOM y guardar un contador interno.

```
import { useRef } from "react";

const inputRef = useRef<HTMLInputElement>(null);
const enfocar = () => inputRef.current?.focus();
// <input ref={inputRef} />
```

8. Errores típicos y cómo corregirlos

Nº	Error	Por qué falla	Corrección
1	`tareas.push(x); setTareas(tareas);`	Muta el array y entrega la misma referencia	`setTareas(prev => [...prev, x])`
2	`onClick={eliminar(id)}`	Se ejecuta al renderizar	`onClick={() => eliminar(id)}`
3	Guardar `longitud` en un segundo estado	Dos datos que pueden contradecirse	`const longitud = texto.length;`
4	`contador = contador + 1`	`contador` es `const`; React no se entera	`setContador(prev => prev + 1)`
5	`map` sin `key` (o con `key` repetida)	React no identifica bien los elementos	`key={item.id}`
6	`useState([])` sin tipo	Se infiere `never[]` y no admite elementos	`useState<Tarea[]>([])`
7	`input` con `value` pero sin `onChange`	Queda bloqueado (solo lectura)	Añadir `onChange` que actualice el estado
8	Formulario sin `preventDefault`	La página se recarga al enviar	`e.preventDefault()` al inicio de `onSubmit`
9	Leer `usuario.edad + 1` con `edad?: number`	Puede ser `undefined`	Comprobar antes o usar `usuario.edad ?? 0`
10	Llamar a `setX` directamente en el render	Provoca un bucle infinito de renders	Hacerlo en un evento o en un `useEffect`
11	Importar `ChangeEvent` de otro sitio	Los tipos de evento vienen de `"react"`	`import { type ChangeEvent } from "react"`
12	Tratar `e.target.value` como número	`value` siempre es `string`	`Number(e.target.value)`

9. Conceptos que se confunden

Conceptos	Diferencia esencial
Props / estado	Las props llegan desde fuera y son de solo lectura. El estado es del componente y se cambia con su setter.
Variable local / `useState`	La variable no provoca render ni persiste. El estado sí.
`type` / `interface`	Ambos describen objetos. `type` también sirve para uniones y literales.
`any` / `unknown`	`any` apaga la comprobación. `unknown` obliga a comprobar el tipo antes de usar el valor.
`target` / `currentTarget`	Elemento que originó el evento / elemento que tiene el manejador.
Mutar / copiar	Mutar cambia el original. Copiar crea una referencia nueva (obligatorio con el estado).
`onClick={f}` / `onClick={f()}`	Pasa la función / la ejecuta al renderizar.
`preventDefault` / `stopPropagation`	Cancela la acción del navegador / detiene la subida del evento.
`||` / `??`	`||` sustituye cualquier valor falso (incluido `0` y `""`). `??` solo `null` y `undefined`.
`useState` / `useRef`	Cambiar el estado provoca render. Cambiar `ref.current` no.
`map` / `filter`	`map` transforma cada elemento y mantiene la longitud. `filter` conserva solo los que cumplen la condición.

Regla rápida cuando «la pantalla no se actualiza». ¿Está el dato en `useState`? ¿Uso el setter? ¿He mutado un array u objeto? ¿Estoy pasando la función o ejecutándola?

10. Ejercicios resueltos (de menor a mayor dificultad)

Ejercicio 1. Contador que no baja de cero

```tsx
import { useState } from "react";

export function ContadorBasico() {
  const [contador, setContador] = useState(0);
  const incrementar = () => setContador(prev => prev + 1);
  const decrementar = () => setContador(prev => Math.max(0, prev - 1));

  return (
    <div>
      <h2>Valor actual: {contador}</h2>
      <button onClick={incrementar}>+</button>
      <button onClick={decrementar}>-</button>
    </div>
  );
}
```

Claves: estado para el número, forma funcional porque depende del anterior, `Math.max(0, ...)` como límite.

Ejercicio 2. Contador de caracteres derivado

```tsx
import { useState, type ChangeEvent } from "react";

export function CajaTextoControlada() {
  const [texto, setTexto] = useState("");
  const caracteres = texto.length;   // derivado: sin segundo estado

  return (
    <div>
      <input
        value={texto}
        onChange={(e: ChangeEvent<HTMLInputElement>) => setTexto(e.target.value)}
      />
      <p>Número de caracteres: {caracteres}</p>
    </div>
  );
}
```

Ejercicio 3. Lista de tareas con formulario controlado

```tsx
import { useState, type FormEvent, type ChangeEvent } from "react";

type Tarea = { id: number; texto: string };

export function ListaTareas() {
  const [texto, setTexto] = useState("");
  const [tareas, setTareas] = useState<Tarea[]>([]);

  const cambiar = (e: ChangeEvent<HTMLInputElement>) => setTexto(e.target.value);

  const anadir = (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    const limpio = texto.trim();
    if (!limpio) return;
    setTareas(prev => [...prev, { id: Date.now(), texto: limpio }]);
    setTexto("");
  };

  return (
    <div>
      <form onSubmit={anadir}>
        <input value={texto} onChange={cambiar} />
        <button type="submit">Añadir</button>
      </form>
      <ul>
        {tareas.map(t => <li key={t.id}>{t.texto}</li>)}
      </ul>
    </div>
  );
}
```

Sobre el `id`: `Date.now()` basta en clase, pero dos altas en el mismo milisegundo repetirían `id`. Alternativas: `crypto.randomUUID()` (el `id` pasa a ser `string`) o un contador guardado en un `useRef`.

Ejercicio 4. Búsqueda con estado elevado

```tsx
import { useState, type ChangeEvent } from "react";

type Producto = { id: number; nombre: string };
type BusquedaProps = { valor: string; alCambiar: (valor: string) => void };
type ListaProps = { productos: Producto[]; filtro: string };

function BarraBusqueda({ valor, alCambiar }: BusquedaProps) {
  const cambiar = (e: ChangeEvent<HTMLInputElement>) => alCambiar(e.target.value);
  return <input value={valor} onChange={cambiar} placeholder="Buscar..." />;
}

function ListaProductos({ productos, filtro }: ListaProps) {
  const visibles = productos.filter(p =>
    p.nombre.toLowerCase().includes(filtro.toLowerCase())
  );
  return <ul>{visibles.map(p => <li key={p.id}>{p.nombre}</li>)}</ul>;
}

export function App() {
  const [filtro, setFiltro] = useState("");
  const productos: Producto[] = [
    { id: 1, nombre: "Camiseta React" },
    { id: 2, nombre: "Libro TypeScript" },
    { id: 3, nombre: "Placa base" },
  ];
  return (
    <main>
      <BarraBusqueda valor={filtro} alCambiar={setFiltro} />
      <ListaProductos productos={productos} filtro={filtro} />
    </main>
  );
}
```

Claves: `App` es el ancestro común y guarda `filtro`; la lista visible es un dato derivado.

Ejercicio 5. Tareas con completar, borrar y contador de pendientes

```tsx
import { useState, type FormEvent } from "react";

type Tarea = { id: number; texto: string; hecha: boolean };

export function Tareas() {
  const [texto, setTexto] = useState("");
  const [tareas, setTareas] = useState<Tarea[]>([]);

  const anadir = (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    const limpio = texto.trim();
    if (!limpio) return;
    setTareas(prev => [...prev, { id: Date.now(), texto: limpio, hecha: false }]);
    setTexto("");
  };

  const alternar = (id: number) =>
    setTareas(prev => prev.map(t => (t.id === id ? { ...t, hecha: !t.hecha } : t)));

  const eliminar = (id: number) =>
    setTareas(prev => prev.filter(t => t.id !== id));

  const pendientes = tareas.filter(t => !t.hecha).length;   // derivado

  return (
    <div>
      <form onSubmit={anadir}>
        <input value={texto} onChange={e => setTexto(e.target.value)} />
        <button type="submit">Añadir</button>
      </form>
      <p>Pendientes: {pendientes}</p>
      <ul>
        {tareas.map(t => (
          <li key={t.id}>
            <input type="checkbox" checked={t.hecha} onChange={() => alternar(t.id)} />
            <span style={{ textDecoration: t.hecha ? "line-through" : "none" }}>{t.texto}</span>
            <button onClick={() => eliminar(t.id)}>Borrar</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Patrón a memorizar: añadir = spread; modificar uno = `map` con `{ ...t, campo }`; borrar = `filter`.

Ejercicio 6. Formulario con validación y mensajes de error

```tsx
import { useState, type ChangeEvent, type FormEvent } from "react";

type Datos = { nombre: string; email: string };
type Errores = { nombre?: string; email?: string };

const validar = (d: Datos): Errores => {
  const errores: Errores = {};
  if (!d.nombre.trim()) errores.nombre = "El nombre es obligatorio";
  if (!d.email.includes("@")) errores.email = "Email no válido";
  return errores;
};

export function FormularioValidado({ alEnviar }: { alEnviar: (d: Datos) => void }) {
  const [datos, setDatos] = useState<Datos>({ nombre: "", email: "" });
  const [errores, setErrores] = useState<Errores>({});

  const cambiar = (e: ChangeEvent<HTMLInputElement>) => {
    const { name, value } = e.target;
    setDatos(prev => ({ ...prev, [name]: value }));
  };

  const enviar = (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    const nuevos = validar(datos);
    setErrores(nuevos);
    if (Object.keys(nuevos).length === 0) alEnviar(datos);
  };

  return (
    <form onSubmit={enviar}>
      <input name="nombre" value={datos.nombre} onChange={cambiar} />
      {errores.nombre && <p>{errores.nombre}</p>}
      <input name="email" value={datos.email} onChange={cambiar} />
      {errores.email && <p>{errores.email}</p>}
      <button type="submit">Enviar</button>
    </form>
  );
}
```

Ejercicio 7. Carrito con callback y total derivado

```tsx
import { useState } from "react";

type Producto = { id: number; nombre: string; precio: number };
type ItemCarrito = Producto & { cantidad: number };

function FilaProducto({ producto, alAgregar }: { producto: Producto; alAgregar: (p: Producto) => void }) {
  return (
    <li>
      {producto.nombre} - {producto.precio} €
      <button onClick={() => alAgregar(producto)}>Agregar</button>
    </li>
  );
}

export function Tienda() {
  const productos: Producto[] = [
    { id: 1, nombre: "Ratón", precio: 15 },
    { id: 2, nombre: "Teclado", precio: 30 },
  ];
  const [carrito, setCarrito] = useState<ItemCarrito[]>([]);

  const agregar = (p: Producto) =>
    setCarrito(prev =>
      prev.some(i => i.id === p.id)
        ? prev.map(i => (i.id === p.id ? { ...i, cantidad: i.cantidad + 1 } : i))
        : [...prev, { ...p, cantidad: 1 }]
    );

  const total = carrito.reduce((suma, i) => suma + i.precio * i.cantidad, 0);

  return (
    <div>
      <ul>{productos.map(p => <FilaProducto key={p.id} producto={p} alAgregar={agregar} />)}</ul>
      <ul>{carrito.map(i => <li key={i.id}>{i.nombre} x {i.cantidad}</li>)}</ul>
      <p>Total: {total} €</p>
    </div>
  );
}
```

Ejercicio 8 (extra). Cargar datos con `fetch`, `cargando` y `error`

```tsx
import { useEffect, useState } from "react";

type Usuario = { id: number; name: string };

export function ListaUsuarios() {
  const [usuarios, setUsuarios] = useState<Usuario[]>([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let cancelado = false;
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(r => {
        if (!r.ok) throw new Error("HTTP " + r.status);
        return r.json() as Promise<Usuario[]>;
      })
      .then(data => { if (!cancelado) setUsuarios(data); })
      .catch(err => { if (!cancelado) setError(err instanceof Error ? err.message : "Error"); })
      .finally(() => { if (!cancelado) setCargando(false); });
    return () => { cancelado = true; };
  }, []);

  if (cargando) return <p>Cargando...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{usuarios.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

`as Promise<Usuario[]>` solo le dice a TypeScript qué esperas; no valida lo que llega de la red.

11. Resumen y checklist

Si el ejercicio pide...	Piensa en...
Guardar y mostrar un valor que cambia	`useState` y su setter
Recibir datos de un padre	Props tipadas
Mostrar un array	`map` y `key` estable
Filtrar con una búsqueda	`filter` y dato derivado
Añadir, modificar, borrar en un array	Spread, `map`, `filter` dentro de `setX(prev => ...)`
Cambiar parte de un objeto de estado	`setObj(prev => ({ ...prev, campo: valor }))`
Procesar un formulario	`onSubmit`, `preventDefault`, validación con `trim()`
Input que cambia	`value` + `onChange`
Hijo que avisa al padre	Callback por props
Dos hermanos comparten dato	Elevar el estado al padre común
Mostrar algo solo a veces	`&&`, ternario o retorno temprano
Datos de una API	`useEffect` con `[]`, estados `cargando` y `error`

Checklist antes del examen

- ☐ Sé crear un proyecto con Vite (`react-ts`) y arrancarlo.
- ☐ Tipo props, estado, eventos y funciones callback.
- ☐ Distingo props de estado y sé usar `children`.
- ☐ No muto arrays ni objetos del estado.
- ☐ Uso `prev => ...` cuando el nuevo valor depende del anterior.
- ☐ No duplico datos derivados en otro estado.
- ☐ Paso funciones a `onClick`, no las ejecuto.
- ☐ Puedo escribir un formulario controlado de memoria.
- ☐ Pongo `key` estable en cada lista.
- ☐ Sé elevar el estado y pasar callbacks.
- ☐ Explico `target`/`currentTarget` y `preventDefault`/`stopPropagation`.
- ☐ Sé hacer narrowing con `undefined`, `typeof`, `?.` y `??`.

12. Simulacros de examen

Simulacro 1 (del material original, con corrección ampliada)

Bloque B. Errores del componente original:

1. `contador = contador + 1`: reasigna una constante de estado. TypeScript lo marca como error y React no re-renderiza. Usa `setContador`.
2. `mensaje` es una variable local que se reinicia en cada render: debe **derivarse** de `contador`.
3. `onClick={sumar()}` ejecuta la función durante el render.
4. La condición `contador > 5` se evaluaría con el valor antiguo dentro del manejador: otro motivo para derivar el mensaje en el render.

```tsx
import { useState } from "react";

function EjemploCorregido() {
  const [contador, setContador] = useState(0);
  const mensaje = contador > 5 ? "Activo" : "Inactivo";   // derivado

  return (
    <div>
      <p>{contador} - {mensaje}</p>
      <button onClick={() => setContador(prev => prev + 1)}>Incrementar</button>
    </div>
  );
}
```

Las preguntas de teoría (1 a 5) y las prácticas 6 y 7 del original equivalen a los ejercicios 2 y 3 del capítulo 10.

Simulacro 2 (nuevo, 45 minutos)

Bloque A. Teoría

1. ¿Cuánto sube `contador` tras pulsar el botón? Justifícalo.

```tsx
const [contador, setContador] = useState(0);
const sumarTres = () => {
  setContador(contador + 1);
  setContador(contador + 1);
  setContador(contador + 1);
};
```

2. ¿Por qué `{lista.length && <Lista />}` puede mostrar un `0` en pantalla?
3. ¿Por qué es mala idea usar el índice del array como `key` en una lista editable?
4. Diferencia entre `any` y `unknown`.
5. ¿Qué significa `[]` como segundo argumento de `useEffect`?
6. ¿Cuándo hay que elevar el estado?

Bloque B. Encuentra al menos cinco errores y corrígelos

```tsx
import { useState } from "react";

function Lista() {
  const [items, setItems] = useState([]);
  const [texto, setTexto] = useState("");

  const anadir = () => {
    items.push({ nombre: texto });
    setItems(items);
  };

  return (
    <div>
      <input onChange={e => setTexto(e.target.value)} />
      <button onClick={anadir()}>Añadir</button>
      <ul>{items.map(p => <li>{p.nombre}</li>)}</ul>
    </div>
  );
}
```

Bloque C. Práctica

7. Crea `Interruptor`: un botón que alterna entre «Encendido» y «Apagado» y muestra el estado actual.
8. Crea `Saludo` con props `nombre: string` y `mensaje?: string` (por defecto `"Bienvenido"`).
9. Crea un componente `Cuadro` que reciba `children` y un `titulo`.

Soluciones del simulacro 2

Inténtalo todo antes de leer esta parte.

Bloque A

1. Sube **1**. Las tres llamadas usan el mismo valor de `contador` de ese render (una «foto»), así que las tres piden establecer `contador + 1`. Con `setContador(prev => prev + 1)` subiría 3.
2. Si `lista.length` es `0`, la expresión `0 && ...` devuelve `0`, y React pinta el número `0`. Se corrige con `lista.length > 0 && ...`.
3. Si los elementos se insertan, borran o reordenan, el índice pasa a otro elemento y React asocia mal el estado interno y el DOM. Usa un `id` estable del dato.
4. `any` desactiva la comprobación y permite todo. `unknown` acepta cualquier valor pero obliga a acotarlo (`typeof`, comprobaciones) antes de usarlo.
5. El efecto se ejecuta una sola vez, al montar el componente (en desarrollo con `StrictMode`, React puede ejecutarlo y limpiarlo una vez extra para comprobar la limpieza).
6. Cuando dos o más componentes necesitan leer o cambiar el mismo dato: el estado va al ancestro común más cercano.

Bloque B. Errores

1. `useState([])` sin tipo: no se sabe qué contiene. Debe ser `useState<Producto[]>([])`.
2. `items.push(...)` muta el estado.
3. `setItems(items)` entrega la misma referencia.
4. El objeto no tiene `id`, y no se normaliza el texto (sin `trim()` ni validación de vacío).
5. El `input` no es controlado (falta `value={texto}`) y el texto no se limpia tras añadir.
6. `onClick={anadir()}` ejecuta la función al renderizar.
7. Falta `key` en el `<li>`.

```tsx
import { useState } from "react";

type Producto = { id: number; nombre: string };

function Lista() {
  const [items, setItems] = useState<Producto[]>([]);
  const [texto, setTexto] = useState("");

  const anadir = () => {
    const limpio = texto.trim();
    if (!limpio) return;
    setItems(prev => [...prev, { id: Date.now(), nombre: limpio }]);
    setTexto("");
  };

  return (
    <div>
      <input value={texto} onChange={e => setTexto(e.target.value)} />
      <button onClick={anadir}>Añadir</button>
      <ul>{items.map(p => <li key={p.id}>{p.nombre}</li>)}</ul>
    </div>
  );
}
```

Bloque C

```tsx
import { useState, type ReactNode } from "react";

export function Interruptor() {
  const [encendido, setEncendido] = useState(false);
  return (
    <div>
      <p>Estado: {encendido ? "Encendido" : "Apagado"}</p>
      <button onClick={() => setEncendido(prev => !prev)}>Alternar</button>
    </div>
  );
}

type SaludoProps = { nombre: string; mensaje?: string };
export function Saludo({ nombre, mensaje = "Bienvenido" }: SaludoProps) {
  return <p>{mensaje}, {nombre}</p>;
}

type CuadroProps = { titulo: string; children: ReactNode };
export function Cuadro({ titulo, children }: CuadroProps) {
  return (
    <section>
      <h2>{titulo}</h2>
      {children}
    </section>
  );
}
```

13. Estrategia para el día del examen

1. **Lee todo el enunciado antes de escribir.** Subraya qué datos existen, cuáles cambian, quién es dueño de cada estado y qué evento dispara cada cambio.
2. **Empieza por los tipos.** Declara `type Tarea`, `type Props`, etc. Te obliga a pensar el modelo de datos y reduce errores después.
3. **Estado mínimo.** Pregunta de cada `useState`: ¿se puede calcular a partir de otro dato? Si sí, no es estado.
4. **Plantilla mental de formulario:** `useState` + `onChange` + `onSubmit` + `preventDefault` + `trim()` + limpiar.
5. **Plantilla mental de lista:** añadir con spread, cambiar con `map`, borrar con `filter`, pintar con `map` + `key`.
6. **Revisa antes de entregar:** ¿hay `key`? ¿pasé funciones a los eventos? ¿mutié algo? ¿limpié el input? ¿importé `useState` y los tipos de `"react"`?
7. **Si te atascas**, escribe primero la versión sencilla que funcione y después añade validaciones y detalles.
8. **En teoría**, responde con la definición corta y un ejemplo de código de una línea; suele puntuar más que una explicación larga sin ejemplo.

Chuleta de una página

```
// ── ESTADO ──────────────────────────────────────────
const [n, setN] = useState(0);                    setN(p => p + 1);
const [lista, setLista] = useState<Tarea[]>([]);
const [obj, setObj] = useState<Datos>({ a: "", b: "" });

// ── LISTAS (siempre copia nueva) ────────────────────
setLista(p => [...p, nuevo]);                                  // añadir
setLista(p => p.filter(t => t.id !== id));                     // borrar
setLista(p => p.map(t => t.id === id ? { ...t, hecha: !t.hecha } : t)); // modificar
setObj(p => ({ ...p, a: "nuevo" }));                           // campo de objeto

// ── PINTAR ──────────────────────────────────────────
{lista.map(t => <li key={t.id}>{t.texto}</li>)}
{cond && <A />}      {cond ? <A /> : <B />}      {lista.length > 0 && <L />}

// ── FORMULARIO ──────────────────────────────────────
const enviar = (e: FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  const limpio = texto.trim();  if (!limpio) return;
  /* actualizar estado */       setTexto("");
};
<form onSubmit={enviar}>
  <input value={texto} onChange={e => setTexto(e.target.value)} />
</form>

// ── PROPS ───────────────────────────────────────────
type Props = { a: string; b?: number; alCambiar: (v: string) => void; children: ReactNode };
function C({ a, b = 0, alCambiar, children }: Props) { /* ... */ }

// ── EVENTOS ─────────────────────────────────────────
onClick={f}   onClick={() => f(id)}   // NUNCA onClick={f()}
ChangeEvent<HTMLInputElement>  FormEvent<HTMLFormElement>  MouseEvent<HTMLButtonElement>

// ── TYPESCRIPT ──────────────────────────────────────
x?.y   x ?? 0   typeof x === "string"   type A = "a" | "b"   useState<T | null>(null)
Partial<T>  Omit<T, "id">  Pick<T, "id">  Record<string, number>

// ── EFECTOS ─────────────────────────────────────────
useEffect(() => { /* al montar */ return () => { /* limpieza */ }; }, []);
```

Referencias oficiales

- **React** (react.dev): *Responding to Events*, *Updating Arrays in State*, *Sharing State Between Components*, *Choosing the State Structure*, *Synchronizing with Effects*, *StrictMode*.
- **TypeScript** (typescriptlang.org/docs): *Everyday Types*, *Narrowing*, *Generics*.
- **Vite** (vite.dev/guide): *Getting Started*.